

Exploring the Effects of Total Variation Loss in Neural Style Transfer

Kaj Hansteen Izora and Colin Aslett
CS 445

1. Introduction

Can AI create unique Art? Neural Style Transfer (NST) proves that it can, creating unique artistic interpretations of original images, blurring the line between human artists and AI. NST operates by merging two images, a content image and a style image, into a target image that has the content of the first with the style of the second image. This technique, powered by neural networks, has created new possibilities in digital art and multimedia applications.

Neural networks require parameters to be heavily tuned to function properly and the same is true for NSTs. Small variations in hyperparameters such as the number of epochs and training data diversity can have a significant impact on the model output. NSTs often incorporate total variation loss (TV loss) to reduce noise and enhance smoothness in the generated image. While the role of individual parameters is well-documented, their interactions in NSTs remains understudied.

This paper will explore how sensitive NSTs are to changes with their parameters. Specifically, we plan to use a pre-trained convolutional neural network (VGG16) to extract content and style features from images. The core of the implementation will involve optimizing a target image to minimize a weighted combination of content loss, style loss and total variation loss, based on differences in feature representations between the content, style, and generated images. We will compare the results of two models, one that is using total variation loss and another that is not.

Our dataset will consist of one style image (*Paysanne Couchant, 1882* by Jules Breton) and 40k content images sourced from cocodataset.org . For evaluation, we will conduct qualitative comparisons between the outputs generated by different configurations and analyze how changes in implementation impact model training, visual fidelity and stylistic accuracy.

2. Background

A 2016 paper from Stanford University Johnson et al. (2016) explored perceptual losses for real-time style transfer and super-resolution. The authors proposed utilizing perceptual loss combined with fast feed-forward networks to generate images faster without losing any quality. Perceptual loss is an alternative to the traditional per-pixel loss functions, MSE for example. Two main methods of perceptual loss were discussed, feature reconstruction loss (FRL) and style reconstruction loss (SRL). Johnson et al (2016) repurposed VGG-16, a deep convolutional neural network architecture originally designed for image classification, as a loss network to compute perceptual losses. VGG-16 helped with feature extraction, extracting the high-level features from an image. The high-level features are then compared to define FRL and SRL. FRL measured the discrepancy between the original features and the reconstructed features generated by the model. SRL measured the discrepancy between the style of the generated image and the style of the style image. Total variation loss, a regularization term used in image processing, was used to reduce noise in image output. TV loss is added to the FRL and SRL to get the total loss, helping smooth out local fluctuations without reducing the quality of the generated image. Perceptual losses were previously shown to produce more realistic textures and edges compared to per-pixel losses. Fast feed-forward networks (FFFN) were primarily using the per-pixel losses, but Johnson et al. (2016) combined FFFN with perceptual losses. Their results for style transfer showed no

decrease in generated image quality but a significant increase in computing speed. When perceptual losses were applied to single image super-resolution the generated image was smoother and more realistic (2016).

Johnson et al (2016) had the following parameters for style transfer: training data of 40,000 and 2 epochs (40K iterations each epoch). In this project, we are exploring to see how the results would change if TV loss was not used and compare. Additionally, we will be comparing the performance of the model with TV loss to the same model without TV loss.

3. Methodology

This project investigates how the inclusion of Total Variation (TV) loss affects the behavior of a neural style transfer (NST) model. We focus specifically on the impact of enabling or disabling TV loss during training, while keeping all other aspects of the model and training process constant.

3.1 Model Architecture

Our NST pipeline is built using two core components:

TransformerNet: A feed-forward neural network used to generate stylized images from content images in a single pass. The network architecture consists of:

- Feature Extraction Layers: Initial convolutional layers to encode the content image.
- Residual Blocks: Multiple layers that preserve spatial structure while allowing stylistic transformations.
- Upsampling Layers: Transposed convolutions to scale the image back to its original size.

VGG16: A pre-trained convolutional network used solely for computing loss. It is not trained during this process. Specific layers of VGG16 are used to extract:

- Content features (from relu2_2)
- Style features (from multiple layers such as relu1_2, relu2_2, relu3_3, relu4_3)

These specific layers are chosen based on their receptive field sizes and level of abstraction.

relu2_2 is used for content features because it captures mid-level structures such as object shapes and edges, preserving the spatial layout of the image without being too sensitive to low-level textures. For style, a combination of layers like relu1_2, relu2_2, relu3_3, and relu4_3 is used because they span both low-level (textures, colors) and high-level (patterns, global structures) features. This multi-layer approach ensures that the style loss captures the full texture hierarchy of the style image.

3.2 Loss Functions

The total loss function used to train TransformerNet combines three components:

$$\mathcal{L}_{\text{total}} = \lambda_c \cdot \mathcal{L}_{\text{content}} + \lambda_s \cdot \mathcal{L}_{\text{style}} + \lambda_{tv} \cdot \mathcal{L}_{\text{TV}}$$

The hyperparameters used are:

$$\lambda_c = 10^5, \quad \lambda_s = 10^{10}, \quad \lambda_{tv} = 10^{-6} \text{ (or 0 when disabled)}$$

Content loss measures the difference between feature representations of the content image and the generated image extracted from a specific layer (e.g., 'relu2_2') of the VGG16 network:

$$\mathcal{L}_{\text{content}} = \frac{1}{2} \sum_{i,j} (F_{ij}^{\text{generated}} - F_{ij}^{\text{content}})^2$$

where F_{ij} represents the activation of the j -th filter at position i in the selected VGG layer.

Style loss is computed as the difference between the Gram matrices of the style image and the generated image, capturing texture and pattern statistics:

$$\mathcal{L}_{\text{style}} = \sum_l w_l \cdot \frac{1}{4N_l^2 M_l^2} \sum_{i,j} (G_{ij}^{\text{generated},l} - G_{ij}^{\text{style},l})^2$$

where G^l is the Gram matrix of features from layer l , N_l is the number of feature maps, and M_l is the spatial size of the feature map.

Total Variation (TV) loss encourages spatial smoothness in the generated image by penalizing differences between neighboring pixels:

$$\mathcal{L}_{\text{tv}} = \sum_{i,j} ((I_{i,j+1} - I_{i,j})^2 + (I_{i+1,j} - I_{i,j})^2)$$

This is the parameter we will be enabling and disabling in our experiments.

3.3 Experiment Design

We conducted a single-variable controlled experiment comparing two configurations:

- TV Model: TV loss included in the total loss function ($\lambda_{\text{tv}}=1\text{e-}6$)
- No TV Model: TV loss excluded ($\lambda_{\text{tv}}=0$)

All other parameters were held constant across both models.

3.4 Dataset & Training

A subset of 40k images from ImageNet and MS COCO were used for training. Images were preprocessed using resizing and random cropping to 256×256 . A single style image ("Paysanne Couchant, 1882") was used throughout training to isolate the effect of TV loss. For all experiments, the following parameters were kept constant:

Parameter	Value	Description
Epochs	5	Number of times the model sees the entire dataset.
TV Loss	Enabled (TV_WEIGHT = 1e-6)	Encourages spatial smoothness in the output.
Content Weight	LAMBDA_CONTENT = 1e5	Scales the importance of preserving content structure.
Style Weight	LAMBDA_STYLE = 1e10	Scales the importance of applying style features.
TV Weight	TV_WEIGHT = 1e-6	Regularizes spatial smoothness in the output.
Learning Rate	LR = 1e-3	Controls the step size for gradient descent.

Image Size	256	All input images are resized to this size.
Style Image	paysanne-couchant-1882.jpg	Same style image used for both models.
Content Images	40k Images from MS COCO	Same Content images used in both models.
Batch Size	12	Number of images processed per training step.
Optimizer	Adam	Optimization algorithm used to update weights.
Architecture	TransformerNet + VGG16	Model and perceptual loss network.
Normalization	mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]	Used for both content and style inputs.
Transforms	RandomCrop & Resize	Data augmentation and preprocessing.
Checkpointing	Every 500 steps	Saves intermediate outputs and model states.

3.5 Evaluation Metrics

To assess model performance, we captured:

- Total Loss, Content Loss, and Style Loss curves throughout training
- Visual comparisons of generated images at checkpoints (every 500 steps)
- Final outputs for both configurations, qualitatively evaluated for smoothness, coherence, and style-content blend

4. Results

This section presents the quantitative and qualitative results comparing neural style transfer models trained with and without Total Variation (TV) loss. Both models were trained under identical conditions, differing only in whether the TV loss term was included in the total loss function. We evaluate how TV loss affects training dynamics, convergence behavior, and the visual quality of generated images. The results include loss curves over time, sample outputs at various training steps, and final image comparisons.



Figure 1: Style Image Used in Training

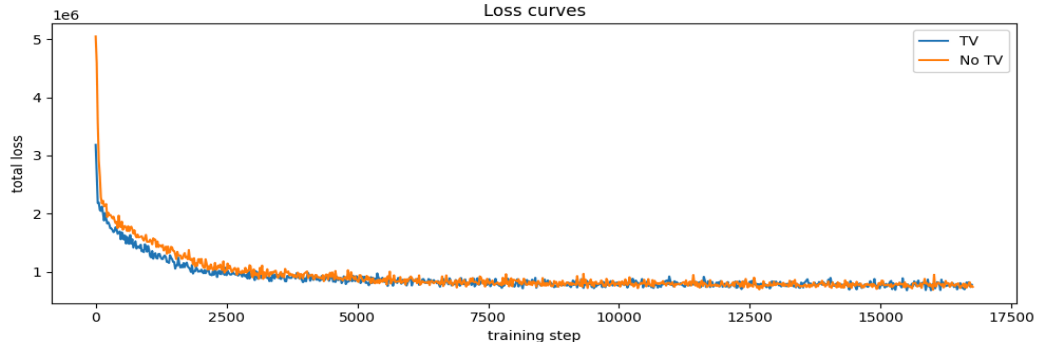


Figure 2: Total Loss Curve of a Model With and Without TV Loss.

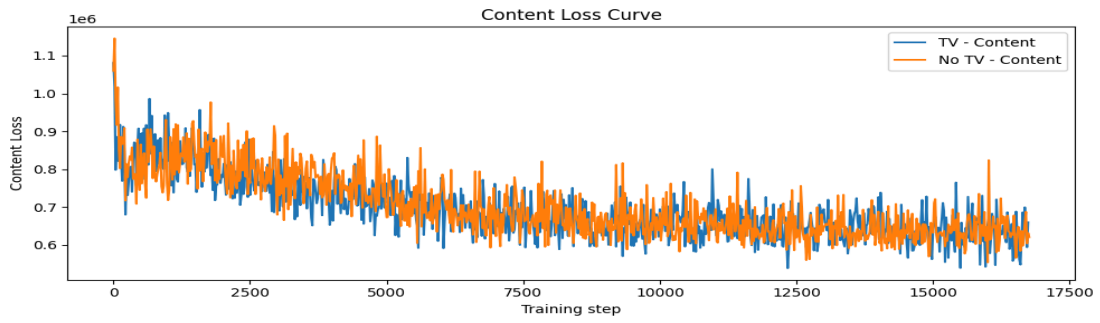


Figure 3: Content Loss Curve of a Model With and Without TV Loss.

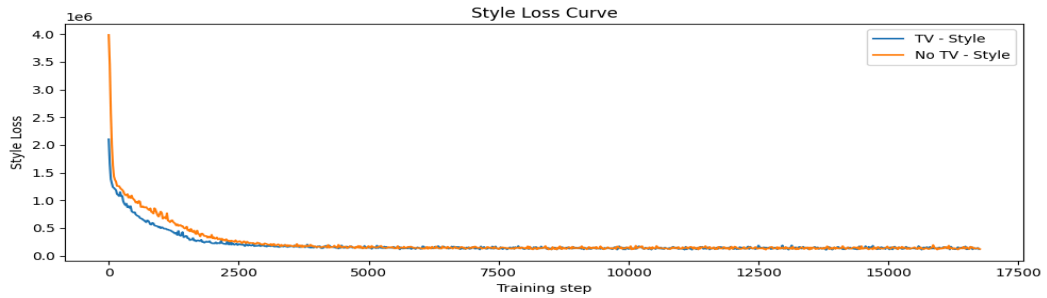


Figure 4: Style Loss Curve of a Model With and Without TV Loss.

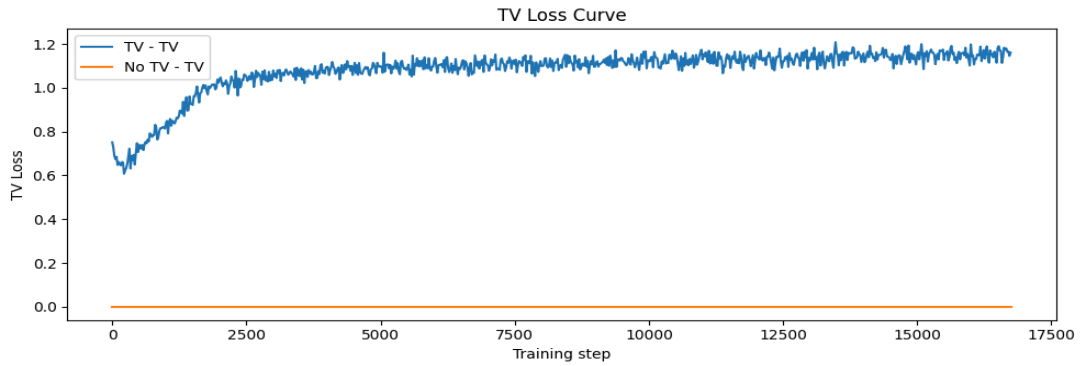


Figure 5: Total Variation Loss Curve of a Model With and Without TV Loss.

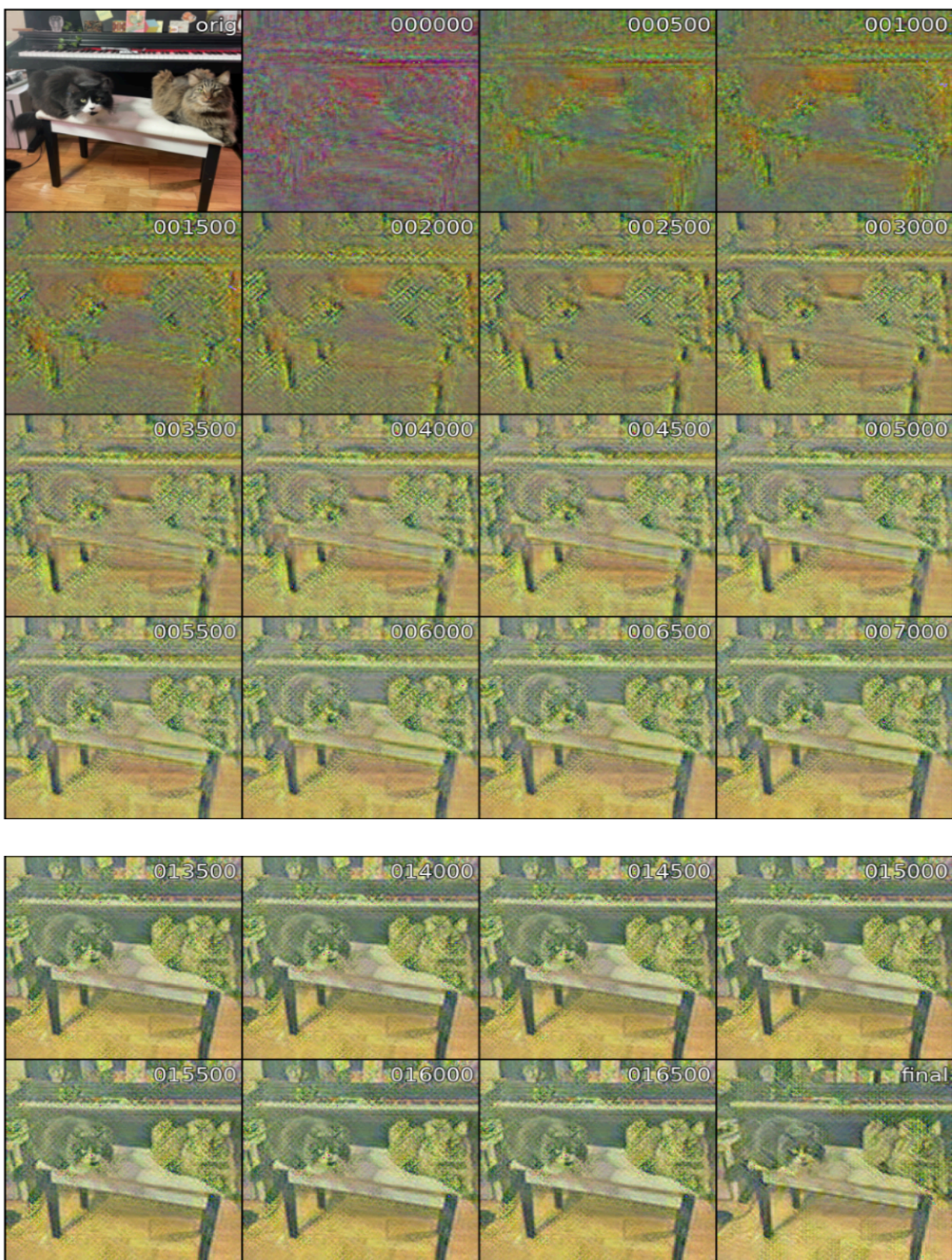


Figure 6: Output Image of Model without TV Loss at Various Training Checkpoints



Figure 7: Output Image of Model with TV Loss at Various Training Checkpoints

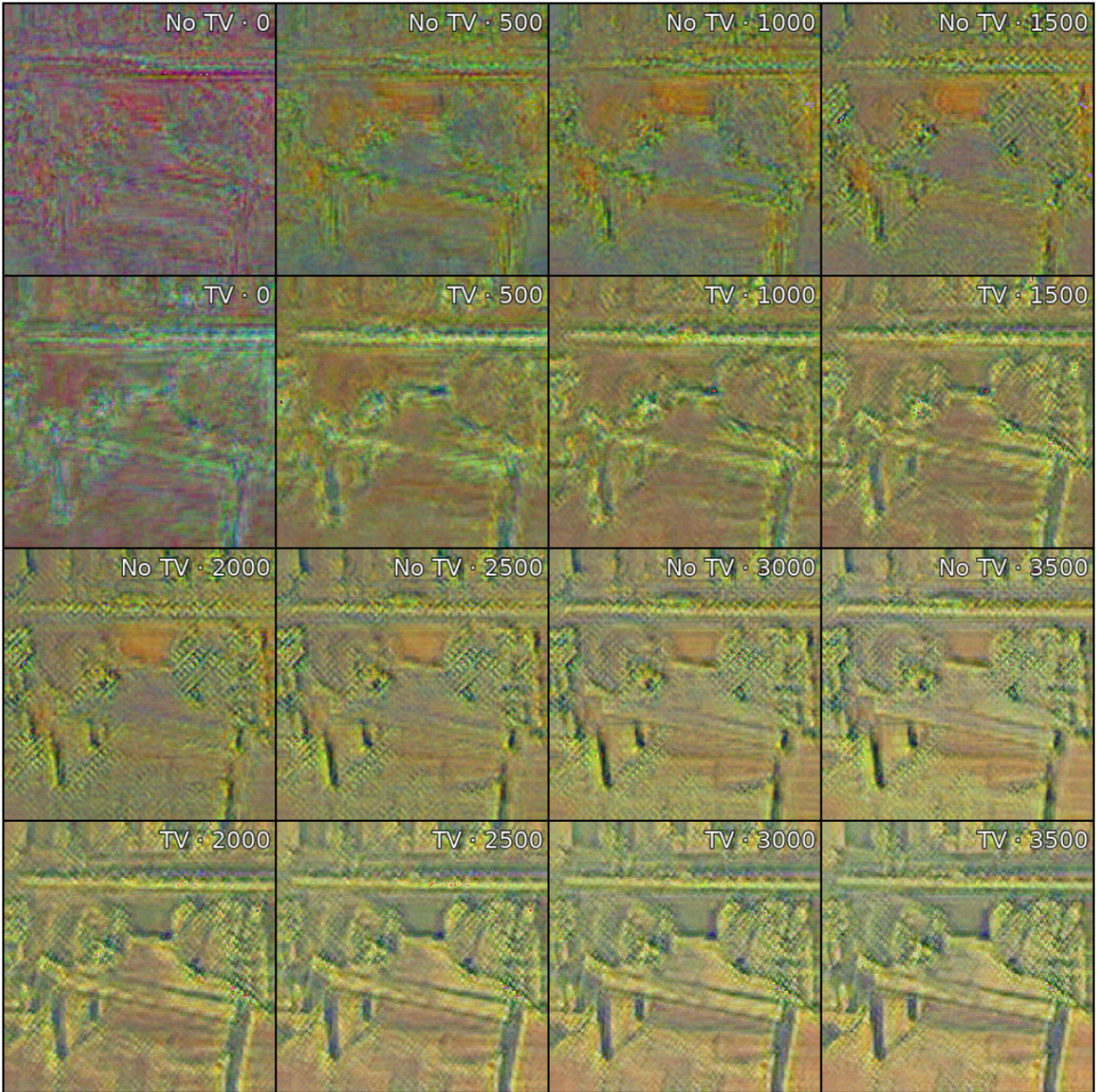


Figure 8: Comparison of Model Outputs Early in Training from TV and No TV Model

Originals



No TV final



TV final



Figure 9: Original Image vs No TV Stylized Image vs TV Stylized Image

Iteration	Content Loss (TV) ($\times 10^5$)	Content Loss (No TV) ($\times 10^5$)	Style Loss (TV) ($\times 10^5$)	Style Loss (No TV) ($\times 10^5$)	TV Loss (TV)	TV Loss (No TV)	Total Loss (TV) ($\times 10^5$)	Total Loss (No TV) ($\times 10^5$)
1	9.0747	9.2837	26.8346	40.8781	0.795	0	35.9093	50.1617
21	9.4437	8.5618	21.6074	32.3306	0.751	0	31.0512	40.8924
41	8.6778	9.2724	18.1289	27.4085	0.703	0	26.8067	36.6809
61	8.8165	8.9836	15.3336	23.8914	0.691	0	24.1501	32.875
81	8.7869	8.6338	13.9105	20.1663	0.697	0	22.6974	28.8002

Table 1: Early Training Loss in TV vs No TV models.

5. Discussion

This section analyzes the results of our experiments, highlighting both general training behavior and the specific effects of Total Variation (TV) loss. We begin by outlining the overall trends observed across all models before comparing the performance of models trained with and without TV loss. Our discussion focuses on both quantitative metrics and qualitative image outputs to assess the impact of TV loss on training dynamics and final results.

5.1 General Analysis

Before comparing the differences between the models with and without TV loss, it’s important to first describe the general behavior observed across all models.

Across all experiments, we observe consistent trends in the total, content, and style loss curves. As shown in Figure 2 (Total Loss Curve), both models experience a rapid decline in total loss during the first few thousand training steps, followed by a slower convergence to a stable value. This behavior is expected, as the model quickly learns large-scale stylization patterns before gradually refining details.

The content loss (Figure 3) exhibits a large initial drop, then slowly decreases and plateaus around 7500 iterations. This trend reflects how quickly the model learns to preserve the

high-level structure of the content image. Meanwhile, the style loss (Figure 4) also drops sharply early on and stabilizes, which is consistent with the network learning to replicate the texture and color distributions of the style image.

One of the most important observations is that style loss is consistently much higher in magnitude than content loss, particularly during the early stages of training. This is due to the much larger weight assigned to style loss ($\lambda_{\text{style}}=1e10$) compared to content loss ($\lambda_{\text{content}}=1e5$), making it the dominant driver of total loss.

Figure 5 (TV Loss Curve) shows that TV loss gradually increases throughout training as the model begins adding more texture and spatial detail to its outputs. This rise is expected. Early outputs are smoother and flatter, while later outputs are more detailed, increasing pixel variation and thus TV loss.

We now turn to our main comparison: evaluating the impact of enabling Total Variation (TV) loss.

5.2 Quantitative and Qualitative Effects of TV Loss

Early in training, TV loss has a significant effect on both quantitative loss metrics and the visual quality of generated images. At step 1 (Table 1), the total loss for the model with TV loss is approximately 3.5 million, compared to 5 million for the model without TV loss. This gap is not due to the TV term itself (TV loss is numerically tiny; less than 1), but instead due to its influence on style loss.

The inclusion of TV loss penalizes noisy and high-frequency outputs, encouraging the model to generate smoother images. Since style loss is computed using Gram matrices (which measure spatial correlations in feature maps), early noisy outputs result in chaotic Gram matrices and high style loss. When TV loss is enabled, these outputs are smoother and better aligned with

the style image's feature correlations, resulting in much lower style loss early in training (2.6 million vs. 4 million after 1 iteration, as shown in Table 1). This explains the large drop in total loss despite TV loss being a minor numerical component.

These quantitative improvements also manifest visually. In Figure 8, we compare output images at various early training steps for each model. In the No TV model, early outputs are highly noisy and distorted, making the content difficult to discern. In contrast, the TV model produces more coherent images much earlier in training, with noticeable structure and reduced artifacting. The style is also blended more cleanly, even from the early iterations.

Despite this early advantage, TV loss has little to no impact on the final results. By around 5000 training steps, both models converge to similar total loss values ($\sim 800,000$), and their final outputs (Figure 9) are nearly indistinguishable. The loss curves also flatten out in unison, indicating that both models reach similar minima in the loss landscape.

Why does this happen? Over time, even without TV loss, the model learns to produce smoother outputs because style loss itself penalizes chaotic textures. The No TV model eventually discovers image representations that align well with the style image, even if it takes longer to do so. As a result, the benefits of TV loss in early training diminish as both models approach the same optimal region.

As for content loss (Figure 3), there is no measurable difference between the TV and No TV models. This is expected. Content loss is computed using deep feature activations from VGG16 (typically from `relu2_2`), which capture high-level image structure but are invariant to small pixel-level changes. Since TV loss only affects local smoothness and not global structure, it does not meaningfully influence content loss.

6. Conclusion

This project investigated the impact of Total Variation (TV) loss in neural style transfer (NST), comparing two models trained under identical conditions—with and without TV loss. Our results show that while TV loss has a noticeable effect early in training—producing smoother, more coherent images and reducing style loss—it does not significantly influence the final outcome after full convergence. Both models ultimately reach similar levels of content and style loss and generate nearly indistinguishable final stylized images.

So, is using TV loss in NST worth it? It depends on your goals. If your priority is faster training, early coherence, or smoother intermediate outputs (e.g., for checkpoints or live demos), then TV loss is a valuable addition. It improves visual quality early on and stabilizes learning. However, if you are optimizing purely for final image quality and plan to train the model to full convergence anyway, TV loss may not be strictly necessary, as its long-term impact diminishes over time. For future work, it would be valuable to explore adaptive or learnable loss weights, multi-style transfer models, or faster real-time applications where early output quality is more critical.

7. Team Contributions

Introduction, background, and methodology done by Colin Aslett.

Results, discussion and conclusion done by Kaj Hansteen Izora.

Both team members contributed to code architecture.

8. Appendix

8.1 Implementation Details

The codebase for this project was adapted from Erik Lindernoren's open-source implementation of neural style transfer, available on GitHub:

<https://github.com/eriklindernoren/Fast-Neural-Style-Transfer>

The implementation uses PyTorch and includes:

- A custom TransformerNet architecture for fast feed-forward stylization.
- Perceptual loss computed using a pre-trained VGG16 model.
- Training scripts configured to allow custom loss weights and checkpointing.
- Image preprocessing using torchvision.transforms, including resizing and cropping to 256×256 pixels.
- Output visualization using PIL and matplotlib.

All experiments were run on a Mac using MPS acceleration where available, and fallback to CPU otherwise.

8.2 Style and Content Image Attribution

The style image used in this project is *Paysanne Couchant, 1882* by Jules Breton. It is in the public domain and was sourced from [WikiArt](#). The content images used for training were sourced from the [MS COCO](#) and [ImageNet](#) datasets. These datasets are widely used in academic research and are publicly available for non-commercial use under their respective licenses.

8.3 Project Repository

Full code for this project can be found in the following git repository:

[Exploring-the-Effects-of-Total-Variation-Loss-in-Neural-Style-Transfer](#)

9. References

1. Johnson, J., Alahi, A., & Fei-Fei, L. (2016).

Perceptual Losses for Real-Time Style Transfer and Super-Resolution.

European Conference on Computer Vision (ECCV).

<https://arxiv.org/abs/1603.08155>

2. Gatys, L. A., Ecker, A. S., & Bethge, M. (2015).

A Neural Algorithm of Artistic Style.

arXiv preprint arXiv:1508.06576.

<https://arxiv.org/abs/1508.06576>

3. Lindernoren, E. (n.d.).

Fast Neural Style Transfer.

GitHub Repository.

<https://github.com/eriklindernoren/Fast-Neural-Style-Transfer>